

Multi-source cross-domain vulnerability detection based on code pre-trained model

Yang Cao^{ID}, Yunwei Dong^{*}

School of Software, Northwestern Polytechnical University, Xi'an, 710129, China

ARTICLE INFO

Keywords:

Vulnerability detection
Deep learning
Multi-source domain adaptation
Code pre-trained model
Cross-language
Cross-project

ABSTRACT

Context: In recent years, deep learning-based vulnerability detection methods have achieved significant success. These methods predict vulnerabilities by automatically learning patterns from code annotated with vulnerability information. However, labeled data is usually concentrated in a few software projects and programming languages. In practice, due to distribution discrepancy in vulnerabilities across different software projects or programming languages, vulnerability detection models trained on limited projects or a specific language often struggle to generalize to new projects or languages. Currently, cross-domain vulnerability detection methods utilize domain adaptation to reduce the distribution discrepancy between the labeled source domain and the target domain being tested. However, the language models used in existing methods limit the expressive power of feature vectors, and they only employ single-source domain adaptation methods.

Objective: To address the limitations of current cross-domain vulnerability detection methods, we propose a new method for Multi-Source cross-domain Vulnerability Detection (MSVD).

Method: MSVD combines two knowledge transfer methods, fine-tuning and domain adaptation. The fine-tuned code pre-trained model extracts code features, generating more meaningful code vector representations. The adversarial-based multi-source domain adaptation method aligns features between multiple source domains and the target domain, leveraging richer knowledge from multiple source domains.

Results: We conducted experiments on real datasets comprising various languages and projects to evaluate the effectiveness of MSVD. Experiment results show that, compared to the baselines in the target domain, MSVD improves F1-score, accuracy, and AUC in the cross-language scenario by 2.95%~112.90%, 4.37%~27.65%, and 4.19%~57.83%, respectively. Additionally, in the cross-project scenario, MSVD achieves the highest F1-score and shows superior performance in terms of accuracy and AUC.

Conclusion: These results indicate that compared to the current state-of-the-art methods, MSVD significantly improves vulnerability detection performance in two cross-domain settings: cross-language and cross-project, when the target domain is unlabeled.

1. Introduction

Software vulnerabilities are defects that affect software security, potentially exploited by attackers to cause system crashes, information leaks, and other issues. With the widespread use of software products and the increasing complexity of software, the number of disclosed vulnerabilities has surged annually. According to statistics, the well-known Common Vulnerabilities and Exposures (CVE) added over 6,000 vulnerabilities in 2016, and by 2023, the number of new vulnerabilities had reached over 29,000 [1]. These software vulnerabilities can lead to significant social and economic impacts. For example, the “Heartbleed” vulnerability exposed hundreds of thousands of the Internet’s secure web servers certified by trusted authorities to risks of private key

leakage or impersonation [2]. The “Ripple20” vulnerability affected hundreds of millions of devices using the Treck TCP/IP stack, allowing attackers to remotely execute code or steal information [3].

To mitigate the impact of software vulnerabilities on system security, an effective measure is to analyze source code and detect vulnerabilities without running the program. These methods are known as static vulnerability detection methods, which can be categorized into program analysis-based and learning-based methods. Program analysis-based methods analyze lexical, syntactic, control flow, data flow, and other information in source code. They use manually defined rules or patterns to detect vulnerabilities. However, manual rule definition is cumbersome, subjective, and costly [4,5]. To reduce manual effort, researchers have proposed learning-based methods to automatically

^{*} Corresponding author.

E-mail addresses: cyang@mail.nwpu.edu.cn (Y. Cao), yunweidong@nwpu.edu.cn (Y. Dong).

<https://doi.org/10.1016/j.infsof.2025.107764>

Received 25 August 2024; Received in revised form 2 March 2025; Accepted 24 April 2025

Available online 22 May 2025

0950-5849/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

learn vulnerability patterns. Various methods have been developed for different types of vulnerabilities. For example, training models to identify code injection attacks [6,7] or detect buffer overflow vulnerabilities [8]. In recent years, due to the advantages of deep learning in extracting deep features, many deep learning-based vulnerability detection methods have emerged and demonstrated superior performance compared to program analysis-based methods [9–11]. Recently, some methods have utilized general linguistic knowledge from large-scale Code Pre-Trained Models (CodePTMs) and achieved better detection results through fine-tuning [12,13]. However, these learning-based methods primarily use supervised learning to learn vulnerability patterns, typically assuming that training and testing data are drawn from the same distribution [14].

In practical vulnerability detection, vulnerabilities in different software projects may have different probability distributions due to factors such as application scenarios, code styles, and programming habits [15]. Similarly, vulnerabilities in different programming languages may exhibit different probability distributions due to syntax structures and programming paradigms. Because of the distribution discrepancy¹ between projects or languages, a vulnerability detection model trained on limited data often struggles to generalize to projects or languages with new probability distributions. Moreover, labeled vulnerability data primarily comes from a few large-scale software projects and mainstream programming languages [16,17], leading vulnerability detection models to face scenarios requiring cross-project or cross-language detection tasks consistently. A set of data and the distribution that generates such data are referred to as a domain. Cross-domain vulnerability detection is used to address the problem of decreased vulnerability prediction accuracy when training data and testing data follow different distributions. In this context, labeled training data is called the source domain, while the unlabeled testing data is referred to as the target domain. Depending on the vulnerability detection task scenario, there are different cross-domain settings. For instance, in this paper, cross-project refers to the source domain and target domain coming from different software projects within the same programming language, while cross-language refers to the source domain and target domain coming from different programming languages.

For the cross-domain vulnerability detection problem, researchers currently use domain adaptation in transfer learning to reduce the distribution discrepancy between the source domain and the target domain. Domain adaptation can align source domain features and target domain features into a joint feature space where their distance is minimized, thereby enabling the learning of an optimal model for the target domain using the source domain, even without labeled data in the target domain. For example, VulGDA minimizes the Maximum Mean Discrepancy (MMD) between the source and target domains to transform features [15], while CDAN uses Generative Adversarial Network (GAN) to reduce the discrepancy between source and target domain in the joint feature space [18]. However, current cross-domain vulnerability detection methods have two main limitations. First, in the feature extraction stage, the training process of existing code language models is limited to datasets of specific projects and languages, which restricts the expressiveness of feature vectors and may affect the accuracy of vulnerability prediction. Second, in the domain adaptation stage, existing methods use a single-source domain adaptation approach. There is often more than one available source domain in the real world. For example, the Bigvul dataset includes many labeled vulnerability data from large-scale open-source projects like Chrome and Linux [16]. Multiple source domains typically contain

richer transferable knowledge, but using a simple single-source domain adaptation method may result in negative transfer due to differences between source domains.

Based on the above analysis, we propose a multi-source cross-domain vulnerability detection method, called MSVD. It is suitable for cross-language and cross-project vulnerability detection scenarios. This method assumes no labeled data in the target domain, using only labeled data from multiple source domains to train and select the model, and then detecting vulnerabilities in the target domain. Specifically, MSVD combines fine-tuning and domain adaptation. It uses a fine-tuned CodePTM to extract code features and then employs an adversarial-based multi-source domain adaptation method to learn domain-invariant features. We demonstrate the effectiveness of MSVD through extensive experiments on real datasets in cross-language and cross-project settings.

The main contributions of this paper are as follows:

- (1) We propose a multi-source cross-domain vulnerability detection method, called MSVD. MSVD operates in an unsupervised domain adaptation manner, enabling it to learn a vulnerability detection model that performs well in the target domain without labeled data. This method combines pre-training fine-tuning and domain adaptation, achieving better performance than existing methods in both cross-language and cross-project settings.
- (2) We extract code features based on a fine-tuned CodePTM, leveraging the code language knowledge learned from large-scale corpora to generate more meaningful code representations, thus aiding the learning of vulnerability patterns. Additionally, we utilize a multilingual CodePTM to achieve vulnerability detection in a cross-language setting for the first time and validate the importance of multilingual CodePTMs for cross-language tasks.
- (3) We employ an adversarial-based multi-source domain adaptation method to learn domain-invariant features, thereby utilizing the knowledge of multiple similar domains to enhance the generalization performance of the vulnerability detection model on new data. This method outperforms existing single-source cross-domain vulnerability detection approaches and can improve model detection performance in the target domain by increasing the number of source domains.

2. Background

This section introduces the technical background necessary for the proposed method. First, we discuss CodePTMs, which learn general language knowledge suitable for software engineering tasks through self-supervised training on large-scale corpora. Then, we explore deep domain adaptation techniques, which are crucial for mitigating the distributional discrepancy between the source and target domains in cross-domain vulnerability detection.

2.1. Code pre-trained models

Pre-training and fine-tuning have become mainstream methods in the field of Natural Language Processing (NLP). As a parameter-based transfer learning method, pre-training models leverage larger corpora and more parameters to learn richer general linguistic knowledge, and then achieve knowledge transfer by sharing model parameters during the fine-tuning process for specific tasks. Considering the similarities between programming languages and natural languages, CodePTMs developed for source code characteristics have recently been applied to various software engineering tasks such as defect detection, code completion, and code generation [19,20].

Currently, the network architecture of CodePTMs is primarily based on Transformers [21] and can be categorized into three types: encoder-based (e.g., CodeBERT [22]), decoder-based (e.g., Codex [23]), and encoder-decoder-based (e.g., CodeT5 [24]). Encoder-based models are

¹ The term “distribution discrepancy” refers broadly to the inconsistency between two distributions, encompassing both explicit metrics (e.g., Maximum Mean Discrepancy, Jensen-Shannon divergence, and Kullback-Leibler divergence) and implicit metrics (e.g., those learned through adversarial learning or geometric transformation).

mainly used for code understanding tasks, while decoder-based and encoder-decoder-based models are primarily used for code generation tasks [19]. These CodePTMs introduce new pre-training tasks that combine the characteristics of code to adapt to software engineering tasks. For example, GraphCodeBERT uses data flow graphs to represent the data dependencies between variables in code snippets and introduces two structure-aware pre-training tasks: Edge Prediction and Node Alignment, to learn code representations through semantic-level structure [25]. Furthermore, CodePTMs are often pre-trained on large-scale datasets in multiple languages, which supports cross-language knowledge transfer.

The goal of source code vulnerability detection is to predict whether a given code snippet is vulnerable. This is a code understanding task, which involves extracting important information related to vulnerabilities from code snippets. According to literature statistics, CodeBERT has become the most widely used CodePTM in vulnerability detection tasks [26]. Additionally, CodeBERT's pure sequence modeling approach can more flexibly adapt to the characteristics of different programming languages without the need to build specialized code parsers for each language. Therefore, we utilize CodeBERT to generate vector representations of source code for cross-language and cross-project vulnerability detection.

2.2. Deep domain adaptation

Training deep neural networks usually relies on large amounts of labeled data. However, supervised learning methods typically assume that training and testing data follow the same distribution. If a model trained on the source data (source domain) is applied to the target data (target domain) with a different but related distribution, distribution discrepancy will lead to a decline in the model's performance on the target data. To achieve good performance on the target domain, it is necessary to collect labeled data in the target domain to train a new model. However, in many real-world applications, such as vulnerability detection, obtaining large-scale labeled data in the target domain is expensive and difficult. In the absence of labeled data in the target domain, domain adaptation mitigates the domain shift by aligning the distributions of the source and target domains. This adjustment enables the learning of a model from the source domain that can generalize well to the target domain. In the domain adaptation field, adversarial-based methods demonstrate advanced performance [27, 28]. Due to their ease of integration with pre-trained models, we employ an adversarial-based multi-source domain adaptation approach to enhance the generalization capability of the vulnerability detection model on new data.

Adversarial-based domain adaptation methods align the distributions through adversarial learning, thereby extracting features that can distinguish source data categories while being unable to distinguish domain categories [28]. Domain-Adversarial Neural Networks (DANN) were the first to introduce the adversarial mechanism [29]. The network comprises a feature extractor, a classifier, and a domain discriminator. The feature extractor extracts features from the input data of both the source and target domains, while the classifier and domain discriminator perform label classification and domain classification tasks based on these features, respectively. During training, the feature extractor and classifier minimize the classification error of the source domain to ensure the learned features are recognizable. Simultaneously, the feature extractor maximizes the domain classification error, engaging in adversarial learning with the domain discriminator to learn domain-invariant features. Since multiple source domains are available in practice, single-source domain adaptation naturally extends to multi-source domain adaptation [30]. Adversarial-based multi-source domain adaptation methods also use discriminators to align the distributions, aiming to confuse the discriminator to distinguish whether features come from the same distribution [31]. For instance, MDAN achieves domain adaptation by aligning the distributions of each source domain and the target domain through multiple domain discriminators [30].

3. Method

To address the two shortcomings of existing cross-domain vulnerability detection methods, we propose MSVD, a multi-source cross-domain vulnerability detection method. MSVD integrates fine-tuning and domain adaptation techniques to enhance the generalization capability of vulnerability detection models on unlabeled target domains. The overview of MSVD is illustrated in Fig. 1.

In the training phase, the input of MSVD consists of labeled training samples from multiple source domains and unlabeled training samples from a target domain. Each sample is a function-level code snippet. First, all training samples are tokenized into token sequences by the tokenizer of the CodePTM, and feature extraction is performed separately on the source and target domain features using a shared feature extractor. The feature extractor is based on a CodePTM and is fine-tuned during training. Then, the domain adaptation part introduces domain discriminators and label classifiers. It utilizes an adversarial-based multi-source domain adaptation method to learn domain-invariant features, thereby improving the generalization performance in the target domain. Finally, the trained vulnerability detection model is obtained.

In the detection phase, the code to be inspected in the target domain is tokenized into token sequences. These token sequences are then fed into the well-trained vulnerability detection model, which outputs a prediction indicating whether the code is vulnerable.

3.1. Problem statement

In this paper, multi-source cross-domain vulnerability detection is formulated as a binary classification problem, where the task is to predict whether a given function is vulnerable. Let X denote the input space and $Y = \{0, 1\}$ denote the label space, where 0 indicates a non-vulnerable sample and 1 indicates a vulnerable sample. $D_{S_1}, D_{S_2}, \dots, D_{S_N}$ and D_T represent $N+1$ different distributions over $X \times Y$, corresponding to the probability distributions of vulnerabilities in various programming languages or software projects. Here, $D_{S_1}, D_{S_2}, \dots, D_{S_N}$ are called source domains, and D_T is called the target domain. Assume there are N labeled datasets $S_1, S_2, \dots, S_N$ and one unlabeled dataset T , where S_i consists of N_i labeled source samples drawn from D_{S_i} , i.e., $S_i = \{(x_{S_i}^j, y_{S_i}^j)\}_{j=1}^{N_i} \sim (D_{S_i})^{N_i}$, and T consists of N_i unlabeled target samples drawn from D_T^X , i.e., $T = \{x_T^j\}_{j=1}^{N_i} \sim (D_T^X)^{N_i}$. D_T^X is the marginal distribution of D_T over X . Our goal is to use the data from multiple source domains to learn a classifier $\eta: X \rightarrow Y$ that generalizes well to the target domain, i.e., η has a low target risk $R_{D_T}(\eta) = \Pr_{(x_T, y_T) \sim D_T}(\eta(x_T) \neq y_T)$.

3.2. Code feature extraction

The vector representation of code contains important features such as syntax and semantics, which are crucial components supporting learning-based vulnerability detection methods. The code language model significantly affects the quality of the vector representations, thereby affecting prediction accuracy. Therefore, in the feature extraction stage, we utilize CodeBERT to generate more precise and meaningful vector representations. CodeBERT is developed based on the RoBERTa architecture and is an encoder-only multilingual and multimodal CodePTM. The training dataset for CodeBERT includes 2.1 million pairs of natural language-program language bimodal samples and 6.4 million unimodal program language samples, covering six programming languages (Python, Java, JavaScript, PHP, Ruby, and Go).

In code feature extraction, the tokenizer and the feature extractor process a single function of source code to obtain a fixed-length representation vector. This representation vector, which serves as the code's feature, is then used for subsequent label classification and domain classification. The source samples and target samples share the same

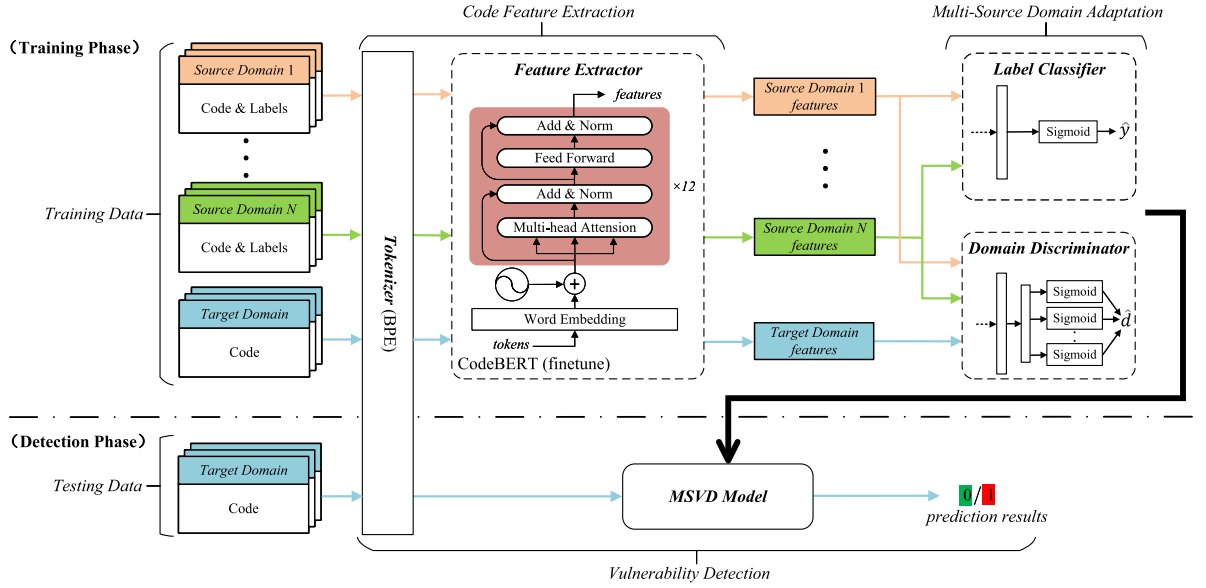


Fig. 1. The overview of MSVD.

tokenizer and feature extractor. For any sample $x \in X$, the feature vector of the code is extracted through the following steps.

First, we use a pre-trained tokenizer $T: X \rightarrow V^m$ to split the function x into a subword sequence $C = ([CLS], c_1, c_2, \dots, c_n, [EOS])$, where V represents the vocabulary of the tokenizer, the length of C is $m = n + 2$, and the token $c_i \in V$. This tokenizer is constructed using the Byte Pair Encoding (BPE) [32], which builds the vocabulary by iteratively merging frequent character pairs. Due to the varying lengths of functions, after splitting x into a sequence, padding or truncation is required to obtain a sequence of length n . Then, special tokens $[CLS]$ and $[EOS]$ are inserted at the beginning and end of the sequence, respectively. For source code containing many custom identifiers, using subwords can effectively represent new custom words in the code. For example, the statement “`retval = mutex_lock_interruptible(&dev->mutex);`” is tokenized into the sequence: (“ $[CLS]$ ”, “`ret`”, “`val`”, “`=`”, “`mut`”, “`ex`”, “`_`”, “`lock`”, “`_`”, “`inter`”, “`,`”, “`,`”, “`,`”, “`[EOS]`”). Here, the custom variable name “`retval`” is split into the more frequently occurring subwords “`ret`” and “`val`”.

Next, we construct a feature extractor $G_f: V^m \rightarrow R^d$ to extract code features from the token sequence. The feature extractor comprises an embedding layer and a stack of 12 Transformer encoder blocks. The parameters of the feature extractor are initialized based on the pre-trained weights of CodeBERT and fine-tuned during the training process. The token sequence C is first processed by the embedding layer into a vector sequence $V = (v_{[CLS]}, v_1, v_2, \dots, v_n, v_{[EOS]})$, where each token in C is encoded into a corresponding vector. Each v_i is obtained by summing the word embedding and the positional encoding. Then, the vector sequence V is input into the stack of Transformer encoder blocks, where each block consists of a multi-head attention layer and a fully connected feed-forward neural network. Finally, the last block of the encoder outputs the contextual vector representation for each token, denoted as $Z = (z_{[CLS]}, z_1, z_2, \dots, z_n, z_{[EOS]})$. The vector representation corresponding to $[CLS]$ can be regarded as the global feature of the entire input sequence and serves as the output of the feature extraction stage.

Let the features of the function $x \in X$ be denoted as $f \in R^d$. The process of code feature extraction is shown in Eq. (1).

$$f = G_f(T(x)) \quad (1)$$

3.3. Multi-source domain adaptation

Common Weakness Enumeration (CWE) provides a hierarchical classification taxonomy for common weaknesses. These weaknesses

could lead to vulnerabilities in certain circumstances. At the highest level, pillar weaknesses describe mistakes from the most abstract perspective. These weaknesses are not specific to any language, technology, framework, or platform but represent abstract themes related to all associated lower-level weaknesses. For instance, “Improper Control of a Resource Through its Lifetime” (CWE-664) indicates that a product does not maintain or incorrectly maintains control over a resource throughout its lifecycle, without implying any specific resource type or access method. As the hierarchy descends, each high-level weakness is refined into more specific security weaknesses, providing more details from dimensions such as behavior, property, resource, language, and technology. The lowest level, variant weakness, typically involves specific languages or technologies. For example, “Stack-based Buffer Overflow” (CWE-121), a variant weakness within CWE-664, refers to the condition where a stack-allocated buffer is overwritten, and this occurs only in C/C++ languages.

Therefore, learning-based vulnerability detection methods can achieve intelligent vulnerability detection by extracting vulnerability features, and utilizing common features at different levels to enable cross-language or cross-project detection. However, due to distribution discrepancy between different projects and languages, a vulnerability detection model trained on limited data often focuses on features specific to the distribution of training data. This limits the model’s ability to generalize to new projects or languages. MSVD addresses this issue by employing an adversarial-based multi-source domain adaptation method during the domain adaptation stage to learn domain-invariant feature representations. This approach leverages knowledge from multiple similar domains to improve the generalization performance of the vulnerability detection model on new data.

The domain adaptation stage comprises a multi-class domain discriminator G_d and a binary label classifier G_c . MSVD aligns data distributions while learning vulnerability patterns based on the source domain features $f_{S_1}, f_{S_2}, \dots, f_{S_N}$ and the target domain feature f_T obtained from the feature extractor.

Label Classifier: The label classifier $G_c: R^d \rightarrow (0, 1)$ is a multi-layer perceptron that performs a binary classification task and outputs classification results through the sigmoid function. The input of the classifier is the features of the source sample, and the output is the probability that the code is vulnerable. MSVD uses cross-entropy loss as the loss function for label classification. For the feature $f_{S_i}^j \in R^d$ of the source sample $(x_{S_i}^j, y_{S_i}^j) \in S_i$, the label classifier loss $L_c^{S_i}$ is shown in Eq. (2).

$$L_c^{S_i}(y_{S_i}^j, y_{S_i}^j) = -[y_{S_i}^j \cdot \log \hat{y}_{S_i}^j + (1 - y_{S_i}^j) \log(1 - \hat{y}_{S_i}^j)] \quad (2)$$

Domain Discriminator: The domain discriminator $G_d: R^d \rightarrow (0, 1)^{N+1}$ comprises a gradient reversal layer and a multi-layer perceptron that performs a multi-classification task, and finally outputs the probability that the sample belongs to each domain through multiple sigmoid functions. The discriminator performs the multi-class domain discrimination task based on code features, with the number of domains being $N+1$. The multi-class discriminator integrates domain discrimination information across different domains, not only focusing on the domain shift between each source domain and the target domain but also considering the discrepancy among the source domains.

During the training process, the domain discriminator and the feature extractor are adversarial to each other. The goal of the domain discriminator is to determine which domain the sample comes from based on the input features, while the feature extractor attempts to extract domain-invariant features to confuse the domain discriminator. To simplify the adversarial training process, a gradient reversal layer [29] is added to the domain discriminator. The gradient reversal layer performs an identity transformation on the features during forward propagation but multiplies the gradient by a negative constant during backpropagation. This gradient reversal mechanism ensures that during the optimization process, the feature extractor not only considers the label classification loss but also contends with the loss from the domain discriminator, making the adversarial training process more efficient.

Referring to MIAN [33], we use multiple binary cross entropy to calculate the domain discrimination loss. Let $\mathbf{1}$ denote a $N+1$ dimensional vector of all ones, and $\mathbf{1}_{[v]}$ denote a $N+1$ dimensional vector with the v -th dimension being 1 and the other dimensions being 0. For source sample $x_{S_i}^j \in S_i$, the domain label is $\mathbf{d}_{S_i}^j = \mathbf{1}_{[i]}$, and the predicted value from the domain discriminator is $\hat{\mathbf{d}}_{S_i}^j \in (0, 1)^{N+1}$. The domain discrimination loss $L_d^{S_i}$ for $x_{S_i}^j$ is shown in Eq. (3). Similarly, for a target sample $x_T^j \in T$, the domain label is $\mathbf{d}_T^j = \mathbf{1}_{[k=N+1]}$, and the domain discrimination loss L_d^T is calculated using Eq. (3). By calculating the binary cross entropy for each dimension of the predicted result, the domain discriminator maximizes the conditional probability of the i th domain while minimizing the conditional probabilities of the other domains.

$$L_d^{S_i}(\hat{\mathbf{d}}_{S_i}^j, \mathbf{d}_{S_i}^j) = -\frac{1}{N+1}[(\mathbf{d}_{S_i}^j)^\top \cdot \log(\hat{\mathbf{d}}_{S_i}^j) + (\mathbf{1} - \mathbf{d}_{S_i}^j)^\top \cdot \log(\mathbf{1} - \hat{\mathbf{d}}_{S_i}^j)] \quad (3)$$

The total loss in the domain adaptation stage consists of the loss of label classification and domain discrimination. To learn feature representations that make multiple source domains indistinguishable from the target domain while containing sufficient information to complete vulnerability prediction, the total loss must consider both the label classification loss for each source domain and the domain discrimination loss for each domain. Given $N+1$ samples (one sample from each domain), the total loss is shown in Eq. (4).

$$L_{total} = \frac{1}{N} \sum_{i=1}^N L_c^{S_i} + \frac{\lambda}{N+1} \left(\sum_{i=1}^N L_d^{S_i} + L_d^T \right) \quad (4)$$

where λ is the weight parameter that adjusts the balance between the classification loss and the discrimination loss. Through this mechanism, the model can focus not only on the label classification task of the source domains but also effectively adapt to the distribution of the target domain through adversarial learning, thereby achieving cross-domain vulnerability detection in a multi-source domain scenario.

3.4. Vulnerability detection

After completing the training, MSVD uses the trained feature extractor and label classifier to predict whether the code is vulnerable. Given a sample $x \in X$ to be detected, MSVD calculates the predicted value $\hat{y}$ of x according to Eq. (5). The value range of $\hat{y}$ is $(0, 1)$. In the end, we set a threshold and map the predicted values to class labels using the piecewise function $\xi: (0, 1) \rightarrow Y$. In this study, the threshold is 0.5.

$$\hat{y} = G_c(G_f(T(x))) \quad (5)$$

4. Experiments

This section presents the experimental evaluation of the proposed MSVD method. First, we introduce the experimental setup, including research questions, datasets, implementation, baseline methods, evaluation metrics, and experimental design. Then, we provide a detailed description and analysis of the experimental results to validate the effectiveness and efficiency of MSVD in cross-language and cross-project scenarios.

4.1. Research questions

To evaluate the performance of MSVD in cross-domain vulnerability detection, we propose the following Research Questions (RQs):

RQ1: How does MSVD perform in the cross-language vulnerability detection? Many high-level weaknesses in the CWE are not specific to any particular programming language. Given the cross-language common characteristics of vulnerabilities, it is expected that a vulnerability detection model can be generalized across different programming languages. Therefore, in this question, we treat each programming language as a domain and investigate the performance of MSVD in cross-language vulnerability detection when the target domain is unlabeled.

RQ2: How does MSVD perform in cross-project vulnerability detection? In the same programming language, due to factors such as application scenarios, development processes, and programming habits, vulnerabilities in different software projects may follow different probability distributions. However, from the perspectives of vulnerability behavior and attributes, these vulnerabilities have common characteristics. Therefore, in this research question, we consider a software project as a domain and investigate the performance of MSVD in cross-project vulnerability detection when the target domain is unlabeled.

RQ3: In cross-project or cross-language vulnerability detection, can increasing the number of source domains improve the detection performance of the model on the target domain? Unlike existing single-source cross-domain vulnerability detection methods, MSVD aims to leverage knowledge from multiple source domains to help the learning in the target domain. However, does increasing the number of source domains improve detection performance in the target domain? Therefore, we investigate MSVD's vulnerability detection performance in the target domain by varying the number of source domains in both cross-language and cross-project vulnerability detection scenarios.

RQ4: In cross-language vulnerability detection, does a CodePTM trained on multiple programming languages perform better? The feature extractor of MSVD is obtained by fine-tuning a CodePTM. Based on the number of programming languages used in the pre-training tasks, CodePTMs can be divided into multilingual models and monolingual models. Cross-language vulnerability detection involves knowledge transfer between languages. Intuitively, a multilingual CodePTM should be better suited for cross-language tasks. Therefore, we investigate whether a multilingual CodePTM has stronger cross-language generalization performance than a monolingual CodePTM in cross-language vulnerability detection.

RQ5: How efficient is MSVD in detecting vulnerabilities? Efficiency is a critical metric for evaluating vulnerability detection methods. If a method requires excessive time for detection, its applicability in real-world scenarios will be significantly constrained. Since MSVD leverages a CodePTM with a substantial number of parameters, it is essential to assess its time overhead to verify its feasibility in practical applications.

4.2. Dataset

In this paper, the data come from three publicly available real-world datasets: Devign [9], BigVul [16], and MegaVul [34]. The code and corresponding vulnerability annotations in these datasets originate from

Table 1
Cross-language vulnerability dataset.

Language	Domain features	Train set			Test set		
		Vul	Non-Vul	Total	Vul	Non-Vul	Total
C	Procedural, Weak typing	9996	19 806	29 802	2538	4911	7449
C++	Multi-paradigm, Strong typing	5688	11 200	16 888	1429	2786	4215
Java	Object-oriented, Strong typing	1896	3839	5735	438	996	1434

Table 2
Cross-project vulnerability dataset.

Project	Domain features	Train set			Test set		
		Vul	Non-Vul	Total	Vul	Non-Vul	Total
Linux	Operating system	2615	5256	7871	645	1321	1966
FFmpeg	Multimedia	3342	6437	9779	793	1647	2440
QEMU	Virtualization	5260	10 565	15 825	1319	2635	3954

real software projects. Devign is constructed by collecting security-related commits and manually labeling the functions. It is a widely used C language vulnerability dataset that includes the open-source software projects FFmpeg and QEMU. BigVul collects data by crawling CVE entries and security information release pages of large software products, annotates vulnerable functions based on code changes, and includes C and C++ code. MegaVul similarly collects and annotates data by crawling CVE entries and covers newly published CVE entries up to 2023, including C, C++, and Java code.

Based on these three datasets, we constructed the dataset used in this paper by filtering samples relevant to cross-language and cross-project tasks. First, we extracted samples from the original datasets according to the programming language or software project of the code. Then, we used tree-sitter [35] to check the code integrity and syntax correctness of the samples. Code integrity requires that the samples include all the code of the original function rather than partial code fragments, and syntax correctness requires that the samples comply with the syntax rules of the corresponding programming language. Finally, we removed duplicate and conflicting data based on the hash values and labels of the code. Duplicate samples mainly resulted from overlaps among the three original datasets, while conflicting samples were due to the same function being modified consecutively.

When constructing the dataset, we selected representative languages and projects based on two dimensions: data scale and domain characteristics. The **cross-language dataset** consists of code from three mainstream programming languages: C, C++, and Java. These languages dominate current vulnerability datasets and learning-based vulnerability detection methods, providing rich vulnerability data resources. Additionally, their significant differences in programming paradigms, memory management, and type systems enable them to represent vulnerability detection scenarios across diverse language characteristics adequately. The **cross-project dataset** comprises three widely used open-source projects in the C language: Linux, FFmpeg, and QEMU. These projects not only possess substantial vulnerability data but also span different application domains such as operating systems, multimedia, and virtualization. This diversity enables rigorous evaluation of the generalization of the method in cross-project scenarios. The details of the cross-language and cross-project datasets are shown in Tables 1 and 2, respectively. Samples from each language or project are divided into training sets and test sets in a ratio of 8:2.

4.3. Experiment implementation

All experiments in this paper were conducted on a machine equipped with two Intel Xeon Gold 6230R CPUs and an NVIDIA GeForce RTX 3090 GPU. MSVD is implemented based on Python 3.8 and PyTorch 1.9.0. We have released the datasets and scripts for this work on GitHub (<https://github.com/Iretinn/msvd>), and details on the implementation and hyperparameters can be found in the publicly available source code.

4.4. Baseline methods and evaluation metrics

In the experiments, we selected the following five baseline methods for comparison with our MSVD approach:

SySeVR [36]: A vulnerability detection method based on Bidirectional Gated Recurrent Units (Bi-GRU), which does not employ domain adaptation.

Devign [9]: A vulnerability detection method based on Gated Graph Neural Networks (GGNN), which does not employ domain adaptation.

CodeBERT [22]: A vulnerability detection method based on CodeBERT, which does not employ domain adaptation.

CodeBERT-mmd: A single-source cross-domain vulnerability detection method based on CodeBERT, utilizing MMD to learn domain-invariant features.

CodeBERT-dann: A single-source cross-domain vulnerability detection method based on CodeBERT, utilizing DANN to learn domain-invariant features.

Existing cross-domain vulnerability detection methods based on domain adaptation, such as VulGDA [15] and CPVD [37], do not provide publicly available source code, and therefore, we did not include them as baselines. Since MSVD employs CodeBERT as the feature extractor and CodeBERT outperforms SySeVR and Devign in our experiments, we developed domain-adapted versions of CodeBERT as baseline methods using MMD and DANN (which are the domain adaptation techniques adopted by VulGDA and CPVD, respectively). This approach allows us to compare the effectiveness of MSVD with single-source cross-domain vulnerability detection methods while avoiding potential performance variations that might arise from selecting other CodePTMs.

To compare the performance of MSVD with baseline methods, the evaluation metrics used in the experiment include Area Under the receiver operating characteristic Curve (AUC), Accuracy (A), Precision (P), Recall (R), and F1-Score (F1). AUC measures the model's classification ability across different thresholds. Accuracy measures the proportion of correctly predicted samples to the total number of samples, as shown in Eq. (6). Precision measures the proportion of actual vulnerabilities among the samples predicted as vulnerabilities by the model, as shown in Eq. (7). Recall measures the proportion of actual vulnerability samples that are correctly predicted as vulnerabilities, as shown in Eq. (8). F1-Score is the harmonic mean of Precision and Recall, as shown in Eq. (9). In the following equations, TP is the number of true positives, FP is the number of false positives, TN is the number of true negatives, and FN is the number of false negatives.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$Precision = \frac{TP}{TP + FP} \quad (7)$$

$$Recall = \frac{TP}{FP + FN} \quad (8)$$

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (9)$$

4.5. Experimental design of research questions

RQ1. In RQ1, we focus on the comparison between MSVD and the baseline methods in cross-language vulnerability detection. The comparative experiments for RQ1 are conducted on the cross-language

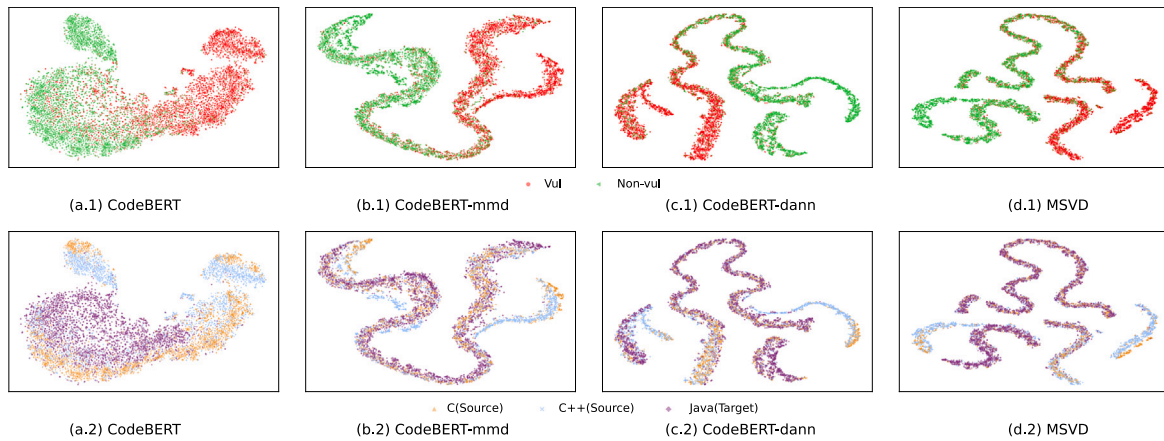


Fig. 2. The t-SNE plots for CodeBERT, CodeBERT-mmd, CodeBERT-dann, and MSVD in C/C++ to Java.

dataset. For the three programming languages in the dataset, two languages are selected as the source domain, and the remaining language is selected as the target domain, conducting a total of 3 experiments. Due to the assumption that the target domain is unlabeled, the source domain test set is used for model selection during training. Specifically, for methods that do not use domain adaptation, we combine the training sets of the two source domains for training, combine the test sets of the two source domains for validation, and finally test on the target domain test set. For single-source domain adaptation methods, we combine the training sets of the two source domains into one, train it together with the target domain training set, then validate on the combined source domain test set, and finally test on the target domain test set. For the multi-source domain adaptation method, we train directly on the two source domain training sets and the target domain training set, validate on the combined source domain test set, and finally test on the target domain test set. It is important to note that during training, the target domain labels are not visible to the model.

RQ2. In RQ2, we focus on the comparison between MSVD and the baseline methods in the cross-project vulnerability detection. The comparative experiments for RQ2 are conducted on the cross-project dataset. We conduct the experiments in the same way as cross-project vulnerability detection, treating the three software projects as the target domain in turn for 3 experiments, and using the source domain test set for model selection.

RQ3. In RQ3, we focus on the impact of the number of source domains on MSVD's performance in the target domain. MSVD can work on one or multiple source domains. Therefore, we train the MSVD model using one source domain and two source domains, respectively, and perform model selection using the corresponding test set of the source domain, and finally test the model in the target domain. The experiments test the impact of the number of source domains in both cross-language and cross-project vulnerability detection. In each cross-domain setting, one project or language is selected as the target domain for three experiments.

RQ4. In RQ4, we investigate whether CodePTMs that are pre-trained in multiple programming languages can achieve better performance in cross-language vulnerability detection. To this end, we compare the performance of MSVD and its variant MSVD-JavaBERT on the cross-language dataset. MSVD and MSVD-JavaBERT initialize the parameters of the feature extractor using CodeBERT [22] and JavaBERT [38], respectively. JavaBERT was developed based on the BERT architecture and adopts the same 12-layer Transformer encoder structure as CodeBERT. Its pre-training dataset contains 3 million Java files.

RQ5. In RQ5, we compare the efficiency of MSVD with three baseline methods (SySeVR, Devign, and CodeBERT) in vulnerability detection. Specifically, we randomly select 1,000 samples from the dataset

and measure the time overhead for each method to complete the detection process. The detection process includes code preprocessing and model inference. To simulate real-world scenarios where samples are analyzed individually, we set the batch size to 1 for all models in the experiment.

4.6. Results

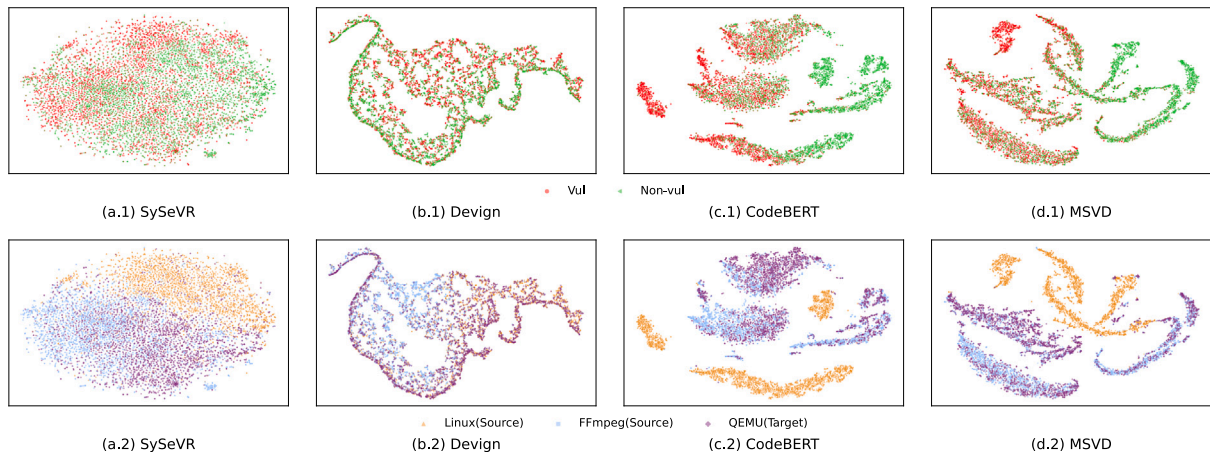
RQ1. Table 3 presents the results of baseline methods and MSVD in cross-language vulnerability detection. By comparing the results on the validation set (from the source domain) and the test set (from the target domain), we can observe the performance changes of the models when transferring from the source domain to the target domain. In the target domain, MSVD shows significantly better performance than other methods across all three experiments, achieving the highest scores in nearly all metrics. In the source domain, CodeBERT performs best, while domain adaptation methods also lead to a slight performance decline in the source domain. In both the source and target domains, SySeVR and Devign exhibit significant performance gaps across various metrics compared to other methods based on CodePTMs. Comparing the F1-Score changes from the source domain to the target domain, Devign's F1-Score decreases by only 0.72 in Java/C++ → C, but its prediction performance in the source domain is inherently poor. As a result, its performance in the target domain also fails to surpass that of methods based on CodePTMs. In C/C++ → Java and C/Java → C++, CodeBERT-mmd and CodeBERT-dann decrease less than CodeBERT, indicating that single-source domain adaptation methods can somewhat mitigate performance loss during cross-language transfer. Although adding domain adaptation to pre-trained models might cause a slight performance decline in the source domain, it can effectively improve the model's performance in the target domain. We believe this is because domain adaptation negatively impacts source domain performance during feature alignment, but enhances the model's generalization ability in the target domain. Considering that most software projects lack labeled vulnerability data, improving the performance of vulnerability detection models in unlabeled target domains is more important in practical applications. Overall, the results in Table 3 show that the multi-source domain adaptation method has stronger generalization ability in cross-language vulnerability detection compared to single-source domain adaptation methods, and also illustrate the feasibility of cross-language vulnerability detection.

Fig. 2 shows the t-SNE plots of CodeBERT, CodeBERT-mmd, CodeBERT-dann, and MSVD when Java is the target domain. The t-SNE plots of each method are shown by vulnerability labels (Fig. 2(a.1) to (d.1)) and domain labels (Fig. 2(a.2) to (d.2)). In Fig. 2(a.2), it can be seen that the features of the target domain (Java) samples generated by CodeBERT are mainly distributed in the lower-left region.

Table 3

Comparison between MSVD and the baseline methods in the cross-language vulnerability detection. The best and second best results for each experiment are marked in **bold** and underline respectively.

Source → Target	Method	Validation results in source domain					Test results in target domain					ΔF1 (%)
		A (%)	F1 (%)	P (%)	R (%)	AUC	A (%)	F1 (%)	P (%)	R (%)	AUC	
C/C++ → Java	SySeVR	69.94	54.24	56.23	52.38	0.7215	69.80	37.33	50.99	29.45	0.6666	16.91
	Devign	66.66	54.88	50.44	60.17	0.6894	64.31	36.71	34.29	39.48	0.5421	18.17
	CodeBERT	<u>79.39</u>	<u>71.18</u>	<u>67.87</u>	74.82	<u>0.8659</u>	72.18	37.95	<u>59.51</u>	27.85	0.7177	33.23
	CodeBERT-mmd	79.70	71.21	68.77	73.83	0.8564	73.18	57.43	55.82	59.13	<u>0.7659</u>	13.78
	CodeBERT-dann	78.66	71.07	65.94	<u>77.06</u>	0.8598	69.69	56.16	50.36	<u>63.47</u>	<u>0.7399</u>	14.91
	MSVD	78.69	71.12	65.96	77.16	0.8664	80.38	69.31	74.03	65.15	0.8556	1.81
C/Java → C++	SySeVR	72.68	44.28	69.95	32.39	0.7505	69.03	30.69	63.65	20.22	0.6716	13.59
	Devign	70.04	56.92	53.91	60.28	0.7207	65.07	38.65	47.07	32.79	0.5876	18.27
	CodeBERT	79.17	<u>67.35</u>	70.93	<u>64.11</u>	0.8550	74.40	56.12	<u>66.99</u>	48.29	0.7653	11.23
	CodeBERT-mmd	76.48	63.84	65.83	61.96	0.8227	73.79	57.88	63.70	53.04	0.7612	5.96
	CodeBERT-dann	78.15	68.90	65.88	72.21	<u>0.8529</u>	<u>74.83</u>	<u>63.47</u>	62.59	64.38	<u>0.7877</u>	5.43
	MSVD	<u>78.89</u>	67.02	<u>70.32</u>	64.01	0.8369	78.10	65.34	69.56	<u>61.57</u>	0.8207	1.68
Java/C++ → C	SySeVR	69.41	50.34	54.31	46.92	0.7078	67.85	40.43	54.82	32.03	0.6787	9.91
	Devign	66.30	49.84	47.66	52.23	0.6455	64.24	49.12	47.27	51.13	0.6294	0.72
	CodeBERT	82.62	74.86	71.70	78.31	0.8885	73.90	60.54	<u>62.44</u>	58.75	<u>0.7811</u>	14.32
	CodeBERT-mmd	<u>81.96</u>	74.34	70.15	<u>79.06</u>	0.8835	72.29	60.60	58.78	62.53	0.7696	13.74
	CodeBERT-dann	<u>81.30</u>	<u>74.55</u>	67.76	82.86	<u>0.8847</u>	70.84	<u>61.48</u>	55.90	<u>68.28</u>	0.7653	13.07
	MSVD	81.77	73.67	<u>70.46</u>	77.18	0.8662	82.00	71.62	74.77	68.72	0.8678	2.05

**Fig. 3.** The t-SNE plots for SySeVR, Devign, CodeBERT, and MSVD in Linux/FFmpeg → QEMU.

Corresponding to the same region in Fig. 2(a.1), there is an overlap between the two types of samples. Overall, Fig. 2(a.1) shows a certain degree of separation between the two types of samples. This result explains why CodeBERT performs well in the source domain (F1-Score of 71.18%) but performs worse in the target domain (F1-Score of 37.95%) in Table 3. Observing the other three domain adaptation-based methods, it can be found that: for vulnerability classification, the samples of the two types exhibit distinct separation (Fig. 2(b.1) to (d.1)); for domain classification, the samples from the three domains are almost entirely overlapping in the feature space (Fig. 2(b.2) to (d.2)). Among them, the best separation between the two types of samples is shown in Fig. 2(d.1), indicating that MSVD can learn feature representations that make multiple source domains indistinguishable from the target domain while still retaining sufficient information for vulnerability prediction.

RQ2. Table 4 presents the results of baseline methods and MSVD in the cross-project vulnerability detection. In the target domain, MSVD achieves the highest F1-Score and relatively high AUC, which is better than the methods without domain adaptation and the methods based on single-source domain adaptation. This demonstrates the effectiveness of combining code pre-trained models with multi-source domain adaptation in cross-project vulnerability detection. In the source domain, CodeBERT exhibits the best performance, achieving the highest AUC in all experiments. Other pre-trained model-based methods (CodeBERT-mmd, CodeBERT-dann, and MSVD) also perform well in the source

domain, while SySeVR and Devign show poorer performance. This indicates that using CodePTMs can significantly enhance performance in vulnerability classification tasks. Comparing the F1-Score changes from the source domain to the target domain, SySeVR and Devign show smaller declines, but their AUC in the target domain is close to 0.5, indicating that their prediction ability is poor after transferring to the target domain. It is worth noting that MSVD exhibits the smallest decline among CodePTM-based methods, indicating that multi-source domain adaptation has stronger knowledge transfer capabilities.

Fig. 3 shows the t-SNE plots of SySeVR, Devign, CodeBERT, and MSVD when QEMU is the target domain. The t-SNE plots of each method are shown by vulnerability labels (Fig. 3(a.1) to (d.1)) and domain labels (Fig. 3(a.2) to (d.2)). From Fig. 3(a.1) and (b.1), it can be seen that the low-dimensional features of SySeVR and Devign have significant overlap areas, indicating that they cannot effectively distinguish between vulnerable and non-vulnerable samples. This is reflected in the lower F1-Score, Precision, and Recall in Table 4 for SySeVR and Devign. In contrast, Fig. 3(c.1) and (d.1) show a clear separation between the two types of samples, which explains why using CodePTMs significantly improves performance, as shown in Table 4. Additionally, we observe that in Fig. 3(c.2) and (d.2), the samples from Linux are distributed outside the clusters of the other two projects. This indicates that the features extracted by CodeBERT for Linux have a significant distribution difference from the other two projects. Moreover, even after using domain adaptation, the feature space of Linux does not align

Table 4

Comparison between MSVD and the baseline methods in the cross-project vulnerability detection. The best and second best results for each experiment are marked in **bold** and underline respectively.

Source → Target	Method	Validation results in source domain					Test results in target domain					ΔF1 (%)
		A (%)	F1 (%)	P (%)	R (%)	AUC	A (%)	F1 (%)	P (%)	R (%)	AUC	
Linux/FFmpeg → QEMU	SySeVR	59.67	44.84	40.49	50.21	0.6103	54.37	39.83	35.57	50.21	0.5310	5.01
	Devign	63.14	47.19	44.32	50.45	0.6084	58.75	38.71	38.38	39.04	0.5111	8.48
	CodeBERT	<u>75.17</u>	68.31	58.54	81.99	0.8291	<u>60.98</u>	53.65	44.43	67.70	0.7440	14.66
	CodeBERT-mmd	74.86	66.38	<u>58.93</u>	75.99	<u>0.8130</u>	63.49	51.17	46.18	57.36	0.6623	15.21
	CodeBERT-dann	75.91	<u>66.54</u>	60.89	73.35	0.7984	60.70	<u>54.89</u>	<u>44.47</u>	<u>71.70</u>	0.7162	11.65
	MSVD	71.33	64.79	54.07	<u>80.81</u>	0.8046	58.78	57.06	<u>43.72</u>	82.11	<u>0.7406</u>	7.73
Linux/QEMU → FFmpeg	SySeVR	60.62	44.17	41.70	46.94	0.5984	54.51	38.33	34.26	43.51	0.5299	5.84
	Devign	61.28	46.30	42.86	50.33	0.6228	52.01	39.44	33.45	48.05	0.5484	6.86
	CodeBERT	75.46	<u>61.89</u>	63.82	60.08	0.8484	68.65	46.62	52.19	42.12	<u>0.6883</u>	15.27
	CodeBERT-mmd	71.72	62.47	55.81	70.93	0.8359	66.23	51.24	48.27	54.60	0.6349	11.23
	CodeBERT-dann	<u>72.99</u>	60.31	<u>58.84</u>	61.86	<u>0.8386</u>	66.72	50.73	48.89	<u>52.71</u>	0.6769	9.58
	MSVD	69.61	59.76	53.29	<u>68.02</u>	0.7720	<u>68.16</u>	56.91	<u>50.79</u>	64.69	0.7405	2.85
FFmpeg/QEMU → Linux	SySeVR	57.21	42.54	38.23	47.96	0.5755	54.48	39.65	35.08	45.58	0.5438	2.89
	Devign	56.43	42.87	37.82	49.48	0.5622	57.20	37.66	36.03	39.44	0.5267	5.21
	CodeBERT	72.05	58.83	57.29	60.46	0.7948	69.79	25.19	67.11	15.50	0.7258	33.64
	CodeBERT-mmd	69.67	<u>60.08</u>	53.13	<u>69.11</u>	0.7811	47.04	37.32	30.47	<u>48.13</u>	0.4294	22.76
	CodeBERT-dann	67.63	62.03	50.63	80.06	0.7657	42.40	20.32	18.58	<u>22.43</u>	0.3937	41.71
	MSVD	<u>70.69</u>	59.17	<u>54.80</u>	64.30	<u>0.7824</u>	<u>68.72</u>	51.38	<u>52.42</u>	50.39	<u>0.7191</u>	7.79

Table 5

Performance of MSVD trained with different numbers of source domains in cross-language vulnerability detection. (C: C, P: C++, J: Java).

Source → Target	A (%)	F1 (%)	P (%)	R (%)	AUC
C → J	69.74	51.99	50.43	53.65	0.7215
P → J	72.04	51.86	54.68	49.32	0.7366
C/P → J	80.38	69.31	74.03	65.15	0.8556
C → P	74.14	62.80	61.29	64.38	0.7879
J → P	65.50	50.10	49.16	51.08	0.6523
C/J → P	78.10	65.34	69.56	61.57	0.8207
J → C	64.38	56.57	48.39	68.09	0.6952
P → C	72.18	61.01	58.39	63.87	0.7620
J/P → C	82.00	71.62	74.77	68.72	0.8678

Table 6

Performance of MSVD trained with different numbers of source domains in cross-project vulnerability detection. (L: Linux, Q: QEMU, F: FFmpeg).

Source → Target	A (%)	F1 (%)	P (%)	R (%)	AUC
L → Q	59.51	36.84	38.40	35.41	0.5344
F → Q	57.74	58.32	43.46	88.63	0.6738
L/F → Q	58.78	57.06	43.72	82.11	0.6782
L → F	59.51	43.22	39.70	47.41	0.5951
Q → F	67.13	51.22	49.47	53.09	0.7281
L/Q → F	68.16	56.91	50.79	64.69	0.7405
F → L	22.33	35.21	24.24	64.34	0.3447
Q → L	64.36	51.01	46.44	56.59	0.6907
F/Q → L	68.72	51.38	52.42	50.39	0.7191

well with the feature spaces of the other two projects. This explains why in [Table 4](#), the performance of CodeBERT, CodeBERT-mmd, and CodeBERT-dann is poor in FFmpeg/QEMU → Linux.

RQ3. [Tables 5 and 6](#) respectively show the performance of MSVD models trained with different numbers of source domains in cross-language and cross-project vulnerability detection. In [Table 5](#), the cross-language vulnerability detection results show a significant performance improvement when the model is trained with two source domains, with F1-Scores 2.54% to 17.32% higher than the best single source domain case. In [Table 6](#), the model trained using the source domain FFmpeg performs slightly better than the model trained with both Linux and FFmpeg only when QEMU is the target domain. In other cases, models trained with two source domains outperform those trained with one. Therefore, increasing the number of source domains enhances MSVD performance in both cross-language and cross-project vulnerability detection.

Table 7

Performance of MSVD using different pre-trained models in cross-language vulnerability detection. (M-J: MSVD-JavaBERT).

Source → Target	Method	A (%)	F1 (%)	P (%)	R (%)	AUC
C/C++ → Java	M-J	71.91	57.30	59.33	55.41	0.7573
	MSVD	80.38	69.31	74.03	65.15	0.8556
C/Java → C++	M-J	73.02	56.73	61.30	52.79	0.7633
	MSVD	78.10	65.34	69.56	61.57	0.8207
Java/C++ → C	M-J	71.09	59.06	55.51	63.10	0.7572
	MSVD	82.00	71.62	74.77	68.72	0.8678

Table 8

Performance of different pre-trained models in in-domain vulnerability detection in different languages.

Language	Method	A (%)	F1 (%)	P (%)	R (%)	AUC
Java	JavaBERT	73.71	60.11	56.02	64.84	0.7841
	CodeBERT	75.45	63.56	58.14	70.09	0.8248
C++	JavaBERT	71.98	59.40	58.38	60.46	0.7715
	CodeBERT	81.49	72.99	72.24	73.76	0.8760
C	JavaBERT	73.85	58.83	63.45	54.85	0.7807
	CodeBERT	76.44	67.84	63.41	72.93	0.8429

Table 9

Time cost in seconds of different methods.

Methods	SySeVR	Devign	CodeBERT	MSVD
Preprocessing Time	31.2	52.6	2.1	2.1
Inference Time	326.2	1.5	19.1	19.6
Total Time	357.4	54.1	21.2	21.7

RQ4. [Table 7](#) presents the results of MSVD and its variant MSVD-JavaBERT in cross-language vulnerability detection. From the table, it can be seen that MSVD outperforms MSVD-JavaBERT in all target domains. Considering that CodeBERT and JavaBERT differ not only in the language type of the pre-training data but also in tokenization and pre-training tasks, we fine-tune CodeBERT and JavaBERT on Java, C, and C++ datasets for in-domain vulnerability detection. The results are shown in [Table 8](#).

In [Table 8](#), CodeBERT outperforms JavaBERT in in-domain vulnerability detection across all three languages. The performance gap is smallest in Java (F1-Score difference of 3.45%) and more significant in C++ and C (F1-Score differences of 13.59% and 9.01%, respectively). This indicates that pre-training on multiple programming languages

Table 10
Comparison of deep learning-based source code vulnerability detection techniques.

Categories	Inputs	Methods	Models	Languages	Domain adaptation
Sequence-Based	Sequence from code	Russell et al. [39]	RNN, CNN	C/C++	–
		Project Achilles [40]	LSTM	Java	–
		LineVul [41]	CodeBERT	C/C++	–
		CDAN [18]	BRNN	C	GAN-based
	Sequence from tree	Feng et al. [42]	Bi-GRU	C/C++	–
		CD-VulD [14]	Bi-LSTM	C/C++	MTLF
Structure-Based	Sequence from graph	AutoVAS [43]	Bi-LSTM	C/C++	–
		SySeVR [36]	Bi-GRU	C/C++	–
	Tree	Dam et al. [44]	Tree-LSTM	C	–
		Devign [9]	GGNN	C/C++	–
		Ghaffarian et al. [45]	GCN, GAT	Java	–
		Reveal [46]	GGNN	C/C++	–
		EnGS2F [47]	GGNN	C++	–
		VulGDA [15]	GGNN	C/C++	MMD
		CPVD [37]	GraphAN	C/C++	DANN
Hybrid	Sequence and Graph	HyVulDect [48]	Bi-LSTM+GGNN	C/C++	–
		CSGVD [12]	Bi-LSTM+GCN	C	–
	Sequence, Tree, and Graph	DeepVD [49]	GRU+Tree-LSTM+GCN	C/C++	–

can improve a CodePTM's ability to detect vulnerabilities in non-pre-trained languages. For cross-language tasks, the smallest F1-Score difference between MSVD-JavaBERT and MSVD is still 8.61% in C/Java $\rightarrow$ C++, demonstrating that multilingual CodePTMs have stronger cross-language knowledge transfer capabilities than monolingual models. Notably, in the Java and C test sets, MSVD achieves higher detection performance through cross-domain training without target domain labels (C/C++ $\rightarrow$ Java and Java/C++ $\rightarrow$ C) than CodeBERT fine-tuned in-domain with labels. This further illustrates the effectiveness of cross-domain vulnerability detection in the absence of target domain labels.

RQ5. Table 9 presents the time cost comparison of vulnerability detection between MSVD and baseline methods. In the code preprocessing stage, both MSVD and CodeBERT require 2.1 s, which is significantly lower than SySeVR and Devign. This discrepancy arises because SySeVR requires normalization of variable and function names, while Devign needs to construct graph representations of the code, leading to longer preprocessing times. In contrast, the pure sequence-based CodePTM avoids complex preprocessing steps, thereby improving overall efficiency. In the model inference stage, since SySeVR is based on the Bi-GRU model and must compute step-by-step, it exhibits the slowest inference speed. In comparison, the inference time of MSVD, which is based on Transformer, is 19.6 s, much lower than that of SySeVR, demonstrating stronger computational efficiency. Additionally, Devign, which is based on GGNN, has the shortest inference time, but its longer preprocessing time results in poor overall efficiency. Overall, MSVD achieves superior computational efficiency while maintaining detection accuracy. Despite using a large-parameter CodePTM, it remains feasible in practical applications.

5. Related work

As shown in Table 10, existing deep learning-based source code vulnerability detection methods can be divided into three main categories based on the type of input used by the models: sequence-based, structure-based, and hybrid representation methods.

Sequence-Based: The source code is composed of character sequences, which naturally leads researchers to leverage natural language processing techniques to handle the source code by representing it as sequences. Russell et al. [39] converted source code directly into token sequences through lexical analysis and used the Recurrent Neural Network (RNN) and the Convolutional Neural Network (CNN) to extract code features in a sentence classification manner. Saccente et al. [40] proposed a function-level vulnerability detection tool, Project Achilles,

which utilized Long Short-Term Memory (LSTM) to process token sequences of Java code. To improve cross-domain generalization, domain adaptation has been introduced into deep learning-based vulnerability detection. Nguyen et al. [18] proposed CDAN based on GAN concepts, using bidirectional RNN (BRNN) as the generator to generate representations of source and target data in a joint space, and reducing distribution differences between the two representations based on the discriminator, ultimately training the classifier using labeled source data. They further proposed Dual-GD-DDAN [50], which employed two generators and two discriminators to handle the source and target domains separately. Since programming languages are more structured than natural languages, some methods incorporated program analysis techniques to generate representations that preserved more code information. For example, Feng et al. [42] obtained node sequences with syntactic features by traversing the Abstract Syntax Tree (AST). CD-VulD [14] used bidirectional LSTM (Bi-LSTM) to extract features and employed the Metric Transfer Learning Framework (MTLF) to learn cross-domain representations. AutoVAS [43] and SySeVR [36] extracted program slices with data and control dependencies from the Program Dependency Graph (PDGs). However, flattening structures like trees and graphs into sequences may still lead to the loss of structural information. In recent years, CodePTMs have been widely applied in vulnerability detection tasks [26]. For instance, LineVul [41] fine-tuned CodeBERT to learn a vulnerability detection model and located vulnerable code lines based on attention scores. Yang et al. [51] addressed label imbalance issues through data sampling during data preparation to enhance fine-tuning performance.

Structure-Based: Structure-based methods directly extract vulnerability features from structural representations such as AST and PDG, which explicitly represent the syntax and semantics behind the code text. AST can effectively represent the syntactic structure of programs. Dam et al. [44] used tree-based LSTM to extract feature vectors from AST, thereby capturing the syntactic features of the code. Since graph structures can represent richer semantic information of code, Graph Neural Networks (GNNs) have been introduced into vulnerability detection. For example, Devign [9] and Reveal [46] utilized GGNN to learn vulnerability features from the Code Property Graph (CPG). Ghaffarian et al. [45] fed Java CPGs into the Graph Convolutional Network (GCN) and the Graph Attention Network (GAT) for vulnerability detection. EnGS2F [47] proposed an enhanced graph structure representation to improve the performance of vulnerability detection. Additionally, structure-based methods have also incorporated domain adaptation to enhance cross-domain generalization. For instance, VulGDA [15] used GGNN to extract feature representations from the CPG and then aligned features by minimizing the MMD between the source and target

domains. CPVD [37] also employed CPG to represent code and used DANN to reduce the distribution difference between the source and target domains.

Hybrid: Hybrid representation methods are based on the idea that combining various representations of code can more comprehensively capture its features. Therefore, some researchers utilize both sequential and structural information of code to enhance model performance in vulnerability detection. For example, HyVulDect [48] used Bi-LSTM to extract token-level features from code context, and used GGNN to extract graph-level semantic features from CPG. These two types of features were then fused and classified using a classifier. CSGVD [12] extracted sequence and graph embeddings using Bi-LSTM and GCN, respectively, and used sequence features as node features in the graph. Ultimately, graph features were used for vulnerability prediction. DeepVD [49] integrated a wider range of code representations to mine code features, including code sequences, long paths on AST, post-dominator trees, exception flow graphs, and calling relations. These features were fused and used for vulnerability prediction. However, hybrid representation methods are often more complex to implement and require handling the challenges of multimodal data fusion.

6. Threats to validity

External validity: The main external threat to this study is the reliability of the data. On the one hand, vulnerable samples and non-vulnerable samples in real-world projects are extremely imbalanced, whereas this study uses a balanced dataset similar to the Devign dataset, with a ratio of vulnerable to non-vulnerable samples of approximately 1:2. However, improving sampling methods can effectively enhance the performance of learning-based vulnerability detection methods on extremely imbalanced data [46]. On the other hand, the accuracy of vulnerability labels can impact the effectiveness of vulnerability detection. The vulnerability data in this study is sourced from BigVul [16] and MegaVul [34]. BigVul is one of the most popular datasets. MegaVul is constructed similarly to BigVul but offers a larger data volume and higher data quality. Both BigVul and MegaVul labeled samples based on code changes. This labeling method may affect accuracy due to irrelevant code changes and other factors [52]. Improving label accuracy requires further research into the construction methods of vulnerability datasets.

Internal validity: First, as a deep learning-based method, the hyperparameter settings can influence the final training results. In response, the feature extractor in our method uses the default hyperparameters specified by CodeBERT, and grid search is employed to select the optimal combination of hyperparameters such as learning rate and domain discriminator loss weight. However, exploring all possible hyperparameter combinations is impractical. Second, our method imposes a length limit on the input code. Truncating functions that exceed this limit may affect the method's vulnerability detection capability. This issue could be addressed by replacing the feature extractor with a CodePTM that supports longer input lengths.

7. Conclusion and future work

This paper presents a cross-domain vulnerability detection method, MSVD, which combines pre-training-finetuning and domain adaptation. MSVD first utilizes a fine-tuned CodePTM to generate feature representations for source and target domain data during the feature extraction phase. Then, in the domain adaptation phase, it employs an adversarial multi-source domain adaptation method to learn domain-invariant features, thereby enhancing the generalization ability of the vulnerability detection model in unlabeled target domains. This method is applicable in both cross-language and cross-project scenarios. Extensive experiments on real-world cross-language and cross-project

datasets demonstrate the effectiveness of MSVD in both cross-domain vulnerability detection scenarios.

Given the unique characteristics of vulnerability data, generalization capability is a critical attribute for vulnerability detection models. Existing methods, including the one proposed in this paper, enhance the generalization ability of models between projects or languages from a domain adaptation perspective. However, the models trained by these methods can only generalize well to a single target domain. For a new target domain, a new model needs to be retrained. Additionally, many small software projects cannot provide sufficient unlabeled data for source and target domain feature alignment. Therefore, future research could explore the application of domain generalization theories in vulnerability detection tasks. This approach would aim to learn a highly generalizable model from several source domains with different data distributions, without relying on any target domain data, thus achieving a “train once, apply everywhere” model. Another important direction for future research is to address scenarios with incomplete or noisy source data, which are common in real-world applications. Techniques such as data augmentation and noise-robust training methods could be explored to enhance the model's robustness and applicability in such settings. This would further bridge the gap between methodological developments and practical deployment of vulnerability detection models.

CRedit authorship contribution statement

Yang Cao: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation. **Yunwei Dong:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Major Program of the National Natural Science Foundation of China (No. 62192733, No. 62192730).

Data availability

I have shared the link to my data/code in the article.

References

- [1] SecurityScorecard, CVE details, 2024, [Online]. Available: <https://www.cvedetails.com/browse-by-date.php>.
- [2] P. Mutton, Half a million widely trusted websites vulnerable to Heartbleed bug, 2014, [Online]. Available: <https://www.netcraft.com/blog/half-a-million-widely-trusted-websites-vulnerable-to-heartbleed-bug>.
- [3] JSOF, Ripple20, 2020, [Online]. Available: <https://www.jsf-tech.com/disclosures/ripple20>.
- [4] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: Proceedings 2018 Network and Distributed System Security Symposium, Internet Society, San Diego, CA, 2018, pp. 1–15, <http://dx.doi.org/10.14722/ndss.2018.23158>.
- [5] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural network, ACM Trans. Softw. Eng. Methodol. 30 (3) (2021) 1–33, <http://dx.doi.org/10.1145/3436877>.
- [6] R. Yan, X. Xiao, G. Hu, S. Peng, Y. Jiang, New deep learning method to detect code injection attacks on hybrid applications, J. Syst. Softw. 137 (2018) 67–77, <http://dx.doi.org/10.1016/j.jss.2017.11.001>, URL <https://www.sciencedirect.com/science/article/pii/S0164121217302571>.
- [7] H. Alnabulsi, R. Islam, I. Alsmadi, S. Bevinakoppa, An innovative method of malicious code injection attacks on websites, Appl. Data Sci. Anal. 2024 (2024) 39–51, <http://dx.doi.org/10.58496/ADSA/2024/005>, URL <https://mesopotamian.press/journals/index.php/ADSA/article/view/397>.

- [8] B.M. Padmanabhuni, H.B.K. Tan, Predicting buffer overflow vulnerabilities through mining light-weight static code attributes, in: 2014 IEEE International Symposium on Software Reliability Engineering Workshops, 2014, pp. 317–322, <http://dx.doi.org/10.1109/ISSREW.2014.26>.
- [9] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Proceedings of the 33rd International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 2019, pp. 10197–10207, <https://dl.acm.org/doi/10.5555/3454287.3455202>.
- [10] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection, IEEE Trans. Dependable Secur. Comput. 18 (5) (2019) 1, <http://dx.doi.org/10.1109/TDSC.2019.2942930>.
- [11] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, H. Jin, VulDeeLocator: A deep learning-based fine-grained vulnerability detector, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2821–2837, <http://dx.doi.org/10.1109/TDSC.2021.3076142>.
- [12] W. Tang, M. Tang, M. Ban, Z. Zhao, M. Feng, CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection, J. Syst. Softw. 199 (2023) 111623, <http://dx.doi.org/10.1016/j.jss.2023.111623>.
- [13] Z. Liu, Z. Tang, J. Zhang, X. Xia, X. Yang, Pre-training by predicting program dependencies for vulnerability analysis tasks, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ACM, Lisbon Portugal, 2024, pp. 1–13, <http://dx.doi.org/10.1145/3597503.3639142>.
- [14] S. Liu, G. Lin, L. Qu, J. Zhang, O. De Vel, P. Montague, Y. Xiang, CD-VulD: Cross-Domain vulnerability discovery based on deep domain adaptation, IEEE Trans. Dependable Secur. Comput. 19 (1) (2022) 438–451, <http://dx.doi.org/10.1109/TDSC.2020.2984505>.
- [15] X. Li, Y. Xin, H. Zhu, Y. Yang, Y. Chen, Cross-domain vulnerability detection using graph embedding and domain adaptation, Comput. Secur. 125 (2023) 103017, <http://dx.doi.org/10.1016/j.cose.2022.103017>.
- [16] J. Fan, Y. Li, S. Wang, T.N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE Summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories, ACM, Seoul Republic of Korea, 2020, pp. 508–512, <http://dx.doi.org/10.1145/3379597.3387501>.
- [17] S. Sakharikar, Systematic review: Analysis of coding vulnerabilities across languages, J. Inf. Secur. 14 (04) (2023) 330–342, <http://dx.doi.org/10.4236/jis.2023.144019>.
- [18] V. Nguyen, T. Le, T. Le, K. Nguyen, O. DeVel, P. Montague, L. Qu, D. Phung, Deep domain adaptation for vulnerable code function identification, in: 2019 International Joint Conference on Neural Networks (IJCNN), IEEE, Budapest, Hungary, 2019, pp. 1–8, <http://dx.doi.org/10.1109/IJCNN.2019.8851923>.
- [19] C. Niu, C. Li, B. Luo, V. Ng, Deep learning meets software engineering: A survey on pre-trained models of source code, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, International Joint Conferences on Artificial Intelligence Organization, Vienna, Austria, 2022, pp. 5546–5555, <http://dx.doi.org/10.24963/ijcai.2022/775>.
- [20] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J.C. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, 2023, <http://dx.doi.org/10.48550/ARXIV.2308.10620>, CoRR, arXiv:2308.10620.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, Curran Associates Inc., Long Beach, CA, USA, 2017, pp. 6000–6010, <https://dl.acm.org/doi/10.5555/3295222.3295349>.
- [22] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, Online, 2020, pp. 1536–1547, <http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139>.
- [23] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F.P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W.H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A.N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, 2021, <http://dx.doi.org/10.48550/arXiv.2107.03374>, CoRR, arXiv:2107.03374.
- [24] Y. Wang, W. Wang, S. Joty, S.C. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 8696–8708, <http://dx.doi.org/10.18653/v1/2021.emnlp-main.685>.
- [25] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S.K. Deng, C.B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, GraphCodeBERT: Pre-training code representations with data flow, 2020, <http://dx.doi.org/10.48550/ARXIV.2009.08366>, CoRR, arXiv:2009.08366.
- [26] X. Zhou, S. Cao, X. Sun, D. Lo, Large language model for vulnerability detection and repair: Literature review and the road ahead, 2024, <http://dx.doi.org/10.48550/ARXIV.2404.02525>, CoRR, arXiv:2404.02525.
- [27] J. Jiang, Y. Shu, J. Wang, M. Long, Transferability in deep learning: A survey, 2022, URL <https://arxiv.org/abs/2201.05867>, arXiv:2201.05867.
- [28] M. HassanPour Zonoozi, V. Seydi, A survey on adversarial domain adaptation, Neural Process. Lett. 55 (3) (2023) 2429–2469, <http://dx.doi.org/10.1007/s11063-022-10977-5>.
- [29] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, V. Lempitsky, Domain-adversarial training of neural networks, J. Mach. Learn. Res. 17 (1) (2016) 2030–2096, <https://dl.acm.org/doi/abs/10.5555/2946645.2946704>.
- [30] H. Zhao, S. Zhang, G. Wu, J.P. Costeira, J.M.F. Moura, G.J. Gordon, Adversarial multiple source domain adaptation, in: Proceedings of the 32nd International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA, 2018, pp. 8568–8579.
- [31] S. Zhao, H. Chen, H. Huang, P. Xu, G. Ding, More is better: Deep domain adaptation with multiple sources, 2024, <http://dx.doi.org/10.48550/ARXIV.2405.00749>, CoRR, arXiv:2405.00749.
- [32] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Berlin, Germany, 2016, pp. 1715–1725, <http://dx.doi.org/10.18653/v1/P16-1162>.
- [33] G.Y. Park, S. Wan Lee, Information-theoretic regularization for multi-source domain adaptation, in: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), 2021, pp. 9194–9203, <http://dx.doi.org/10.1109/ICCV48922.2021.00908>.
- [34] C. Ni, L. Shen, X. Yang, Y. Zhu, S. Wang, MegaVul: A C/C++ vulnerability dataset with comprehensive code representations, in: 21st IEEE/ACM International Conference on Mining Software Repositories, MSR 2024, Lisbon, Portugal, April 15–16, 2024, ACM, 2024, pp. 738–742, <http://dx.doi.org/10.1145/3643991.3644886>.
- [35] M. Brunsfeld, Tree-sitter, [Online]. Available: <https://tree-sitter.github.io/tree-sitter>.
- [36] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2244–2258, <http://dx.doi.org/10.1109/TDSC.2021.3051525>.
- [37] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Trans. Softw. Eng. (2023) 1–17, <http://dx.doi.org/10.1109/TSE.2023.3285910>.
- [38] N.T. De Sousa, W. Hasselbring, JavaBERT: Training a transformer-based model for the java programming language, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), 2021, pp. 90–95, <http://dx.doi.org/10.1109/ASEW52652.2021.00028>.
- [39] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, M. McConley, Automated vulnerability detection in source code using deep representation learning, in: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), 2018, pp. 757–762, <http://dx.doi.org/10.1109/ICMLA.2018.00120>.
- [40] N. Saccente, J. Dehlinger, L. Deng, S. Chakraborty, Y. Xiong, Project achilles: A prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network, in: 2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW), (ISSN: 2151-0830) 2019, pp. 114–121, <http://dx.doi.org/10.1109/ASEW.2019.00040>, URL <https://ieeexplore.ieee.org/document/8967427?arnumber=8967427>.
- [41] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: 2022 IEEE/ACM 19th International Conference on Mining Software Repositories, MSR, 2022, pp. 608–620, <http://dx.doi.org/10.1145/3524842.3524852>.
- [42] H. Feng, X. Fu, H. Sun, H. Wang, Y. Zhang, Efficient vulnerability detection based on abstract syntax tree and deep learning, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, Toronto, ON, Canada, 2020, pp. 722–727, <http://dx.doi.org/10.1109/INFOCOMWKSHPS50562.2020.9163061>.
- [43] S. Jeon, H.K. Kim, AutoVAS: An automated vulnerability analysis system with a deep learning approach, Comput. Secur. 106 (2021) 102308, <http://dx.doi.org/10.1016/j.cose.2021.102308>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0167404821001322>.
- [44] H.K. Dam, T. Pham, S.W. Ng, T. Tran, J.C. Grundy, A. Ghose, T. Kim, C.J. Kim, Lessons learned from using a deep tree-based model for software defect prediction in practice, in: Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26–27 May 2019, Montreal, Canada, IEEE / ACM, 2019, pp. 46–57, <http://dx.doi.org/10.1109/MSR.2019.00017>.
- [45] S.M. Ghaffarian, H.R. Shahriari, Neural software vulnerability analysis using rich intermediate graph representations of programs, Inform. Sci. 553 (2021) 189–207, <http://dx.doi.org/10.1016/j.ins.2020.11.053>, URL <https://www.sciencedirect.com/science/article/pii/S0020025520311579>.

- [46] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: are we there yet? *IEEE Trans. Softw. Eng.* 48 (9) (2022) 3280–3296, <http://dx.doi.org/10.1109/TSE.2021.3087402>.
- [47] P. Xiao, Q. Xiao, X. Zhang, Y. Wu, F. Yang, Vulnerability detection based on enhanced graph representation learning, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 5120–5135, <http://dx.doi.org/10.1109/TIFS.2024.3392536>.
- [48] W. Guo, Y. Fang, C. Huang, H. Ou, C. Lin, Y. Guo, HyVulDect: A hybrid semantic vulnerability mining system based on graph neural network, *Comput. Secur.* 121 (2022) 102823, <http://dx.doi.org/10.1016/j.cose.2022.102823>, URL <https://www.sciencedirect.com/science/article/pii/S0167404822002176>.
- [49] W. Wang, T.N. Nguyen, S. Wang, Y. Li, J. Zhang, A. Yadavally, DeepVD: Toward class-separation features for neural network vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, Melbourne, Australia, 2023, pp. 2249–2261, <http://dx.doi.org/10.1109/ICSE48619.2023.00189>, URL <https://ieeexplore.ieee.org/document/10172789/>.
- [50] V. Nguyen, T. Le, O. De Vel, P. Montague, J. Grundy, D. Phung, Dual-component deep domain adaptation: A new approach for cross project software vulnerability detection, in: *Advances in Knowledge Discovery and Data Mining*, vol. 12084, Springer International Publishing, Cham, 2020, pp. 699–711, http://dx.doi.org/10.1007/978-3-030-47426-3_54.
- [51] X. Yang, S. Wang, Y. Li, S. Wang, Does data sampling improve deep learning-based vulnerability detection? Yeas! and Nays!, in: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023, IEEE, 2023, pp. 2287–2298, <http://dx.doi.org/10.1109/ICSE48619.2023.00192>.
- [52] R. Croft, M.A. Babar, M.M. Kholoosi, Data quality for software vulnerability datasets, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, Melbourne, Australia, 2023, pp. 121–133, <http://dx.doi.org/10.1109/ICSE48619.2023.00022>.